VS AgentCore Multi-Agent Platform

Detailed System Flows & Harness Engineering

Clinical Trial Research Agent Platform

AWS Bedrock AgentCore | LangGraph | Pinecone | Cohere Rerank | Neo4j | GPT-4o | May 2026

Prudhvi Akella@2026

1. What We Built and Why

The VS AgentCore Multi-Agent Platform is a production-grade clinical trial research system that answers complex pharmaceutical and clinical evidence questions by orchestrating five specialised AI agents. The core problem it solves is this: clinical trial data lives across two fundamentally different stores — raw protocol text in PDFs (vectorised into Pinecone) and structured trial metadata in a relational knowledge graph (Neo4j). Neither store alone can answer the full range of questions a pharmaceutical researcher asks. The platform routes each question to the right store, retrieves grounded evidence, and synthesises a cited answer — never fabricating a fact the retrieval layer did not produce.

The system is deployed as a course demonstration for Vidya Sankalp's Applied GenAI Engineering programme. Students see a real production deployment — not a notebook — with real AWS infrastructure, real clinical trial documents, and a real evaluation harness measuring every component of the retrieval pipeline.

1.1 The Five Agents — Responsibilities and Data Sources

Agent	Responsibility	LLM	Primary Data Source
Supervisor	Orchestrates the full conversation. Reads the user question, applies the 5-layer middleware stack (cache, memory, HITL, guardrail, tracing), decides which sub-agent to call, and synthesises the final answer. Never answers clinical questions from its own training data.	GPT-4o-2024-08-06	Reasoning only — no retrieval
Research Agent	Retrieves clinical evidence from protocol PDFs. Handles efficacy results, adverse events, dosing schedules, endpoint definitions, and protocol procedures. Runs the 4-stage RAG pipeline (Pinecone → Cohere → GPT-4o-mini → GPT-4o).	GPT-4o-2024-08-06 (ReAct) + GPT-4o-mini (Stage 3 compression)	Pinecone: clinical-trials-index, namespace clinical-trials — 5,772 chunks from 29 NCT IDs
Knowledge Agent	Answers structured metadata questions using Cypher queries against Neo4j. Handles enrollment counts, trial status, phase, sponsors, CROs, site locations, patient eligibility criteria, outcome measure definitions, MeSH terms, and geographic distribution.	GPT-4o-2024-08-06	Neo4j: 25 Trial nodes, 10 node types, 11 relationship types, 1,149 LOCATED_AT relationships

Chart Agent	Generates Chart.js visualisation configurations from numerical data returned by the Research or Knowledge agent. Does not retrieve data itself — it receives structured numbers and produces a chart spec that the UI renders.	GPT-4o-2024-08-06	Data passed from Research/Knowledge agent
Safety Agent	Deployed as a live AgentCore runtime but intentionally excluded from the Supervisor's agent registry. Output safety is handled at the middleware level by Bedrock Guardrail (OutputGuardrailMiddleware) which is architecturally more appropriate — it checks every response before it leaves the Supervisor.	N/A	Bedrock Guardrail service

📌 **DESIGN PRINCIPLE**	The Supervisor is a pure router. It does not answer clinical questions from GPT-4o's training data. Every factual claim in an answer must come through the Research or Knowledge agent's retrieval step. This eliminates one entire class of hallucination — Supervisor-level fabrication.

1.2 Production Infrastructure

Component	Value / Details
AWS Account / Region	107282186797 / us-east-1
Supervisor AgentCore Runtime	vs_agentcore_ma_supervisor-noB24q7868 (arm64 microVM)
Research AgentCore Runtime	vs_agentcore_ma_research-y5bSF5A4mW (arm64 microVM)
Knowledge AgentCore Runtime	vs_agentcore_ma_knowledge-Ge1zkiFSQm (arm64 microVM)
Chart AgentCore Runtime	vs_agentcore_ma_chart-b1au1U2ioF (arm64 microVM)
Safety AgentCore Runtime	vs_agentcore_ma_safety-3BSTbuGcFt (arm64 microVM, not in registry)
Platform UI	ECS Fargate amd64, FastAPI + SSE streaming
Application Load Balancer	vs-agentcore-ma-alb-550721339.us-east-1.elb.amazonaws.com (idle timeout: 300s)
MCP Gateway	Bedrock AgentCore MCP Gateway vs-agentcore-ma-mcp — routes tool calls to 5 Lambda functions

Lambda Tools (5)	vs-agentcore-ma-search-tool, summariser-tool, graph-tool, hitl-tool, chart-tool (all x86_64)
Pinecone Index	clinical-trials-index, namespace: clinical-trials (5,772 chunks), cache_pharma (semantic cache)
Neo4j	Aura cloud — 25 trials, 10 node types, 11 relationship types
RDS PostgreSQL	LangGraph conversation checkpointing — one checkpoint row per (thread_id, step)
DynamoDB	vs-agentcore-ma-traces — full observability trace per request, TTL 30 days
Bedrock Prompt Management	Supervisor: 66UDJ8VW3W (v18), Research: 44F06K0CNE (v19/DRAFT)
Bedrock Guardrail	Applied at OutputGuardrailMiddleware in the Supervisor middleware stack
SSM Prefix	/vs-agentcore-multiagent/prod — all shared config and agent ARNs
Observability	DynamoDB traces + CloudWatch (Watchtower lazy handler) + LangSmith run trees

2. Complete Request Flow — Step by Step

This section traces a single user request from the browser POST through every component to the final SSE done event. Every component, every data transformation, and every design decision is explained. The example query is: 'What was the annualized relapse rate reduction in the CLARITY trial?'

2.1 Step 1 — HTTP POST Arrives at FastAPI Platform (ECS Fargate)

The browser or curl client sends an HTTP POST to the ALB endpoint. The FastAPI platform validates the X-API-Key header against the value stored in Secrets Manager at /vs-agentcore-multiagent/prod/platform_api_key. If invalid it returns 401 immediately. The request body carries three fields:

```
POST http://vs-agentcore-ma-alb-550721339.us-east-1.elb.amazonaws.com/api/v1/clinical-trial/chat
X-API-Key: <PLATFORM_API_KEY>
Content-Type: application/json

{
  "message":   "What was the annualized relapse rate reduction in the CLARITY trial?",
  "thread_id": "c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b",
  "domain":    "pharma"
}
```

The platform opens an SSE streaming response immediately (status 200, Content-Type: text/event-stream) before calling AgentCore. This is critical: ALB has a default idle timeout of 60 seconds. Clinical trial queries can take 30-90 seconds. The Terraform infra step sets ALB idle_timeout.timeout_seconds to 300 to prevent premature connection drops.

2.2 Step 2 — SigV4-Signed AgentCore Invocation

The platform calls the Supervisor's AgentCore runtime via the Bedrock AgentCore HTTP endpoint. The runtime ARN is read from SSM at /vs-agentcore-multiagent/prod/agents/supervisor/runtime_arn. The request is signed with AWS SigV4 using the ECS task role credentials. AgentCore enforces IAM auth — unsigned requests are rejected with 403.

```
POST https://bedrock-agentcore.us-east-1.amazonaws.com
    /runtimes/arn%3Aaws%3Abedrock-agentcore%3Aus-east-
1%3A107282186797%3Aruntime%2F.../invocations
X-Amzn-Bedrock-AgentCore-Runtime-Session-Id: c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b
Authorization: AWS4-HMAC-SHA256 Credential=.../aws4_request, SignedHeaders=..., Signature=...

{"message": "What was the ARR reduction...", "session_id": "c73e7373...", "domain": "pharma"}
```

2.3 Step 3 — AgentCore MicroVM Warm/Cold Start Decision

AgentCore maps the X-Amzn-Bedrock-AgentCore-Runtime-Session-Id header to a container instance. Two paths exist:

Cold Start (first call for this thread_id or idle timeout exceeded)

The container starts from scratch. The _ensure_agent() function in app.py runs once and caches the agent globally in the _agent variable. Cold start sequence:

1. Reads OpenAI API key from Secrets Manager at /vs-agentcore-multiagent/prod/openai
2. Attaches Watchtower CloudWatch log handler lazily (does not block if CloudWatch is unavailable)
3. Calls get_mcp_tools() — connects to the MCP Gateway and discovers all 5 tool definitions via the MCP protocol handshake
4. Calls build_agent() — builds the LangGraph ReAct graph with the 5-layer middleware stack, connects to RDS Postgres for checkpointing, connects to Pinecone for semantic cache and episodic memory
5. Reads the system prompt from Bedrock Prompt Management — SSM path /{AGENT_NAME}/prod/bedrock/prompt_id and /{AGENT_NAME}/prod/bedrock/prompt_version. SSM is read fresh on every invocation (not cached) so prompt version changes take effect immediately without a redeploy.

⚠ CRITICAL LESSON	AgentCore microVMs stay warm for 15 minutes after last use. The global _agent variable persists across all warm invocations. Do NOT use @lru_cache on SSM reads, Bedrock prompt fetches, or any configuration that changes between deploys. If you cache the prompt version at cold start, the container will serve that version forever — even after you update SSM and push a new prompt. This is the exact bug that caused the ResourceNotFoundException: PromptVersion 6 doesn't exist failure during production debugging.

Warm Start (subsequent calls within 15 minutes)

_agent is already built. _ensure_agent() returns immediately. The container jumps straight to handler() processing. Warm starts add <200ms overhead versus the 3-8 second cold start.

2.4 Step 4 — Middleware Before-Agent Chain

Before the Supervisor LLM is invoked, the 5 middleware layers execute in outermost-to-innermost order. Each layer has a before_agent hook that can either pass control to the next layer or short-circuit (return early with a cached answer).

Layer	Middleware	Before Agent Action	Short-Circuit?

1 — Outermost	TracerMiddleware	Records request start time using time.time(). Stores run_id (= thread_id) and context dict (user_id, session_id, domain) for the after_agent hook to use. Runs the before_model / after_model hooks on every LLM call to record per-turn token usage and latency.	No — always passes through
2	SemanticCacheMiddleware	Embeds the user query with text-embedding-3-small via OpenAI. Queries Pinecone namespace 'cache_pharma' with top_k=1 and cosine similarity threshold 0.97. Skips multi-turn conversations (len(human_messages) > 1) because cached answers lack the conversational context needed for follow-up questions.	YES — on cache HIT, injects cached AIMessage and sets jump_to='end', bypassing all remaining layers and the LLM entirely
3	EpisodicMemoryMiddleware	Queries Pinecone namespace 'episodic__<user_id>' for the top 3 past Q&A pairs most semantically similar to the current question. If found, injects them as a SystemMessage: 'Based on past conversations with this user: [Q1: ... A1: ...]'. Limited to 3 hits to cap token overhead at ~300 tokens regardless of session history length.	No — always passes through
4	HITLMiddleware	No action on fresh requests. On resume requests (payload.resume = True), injects a SystemMessage preventing a second HITL call: 'The user has already answered the clarifying question — do not call ask_user_input again.'	No
5 — Innermost	OutputGuardrailMiddleware	No action before agent. Guardrail checks the output, not the input.	No

2.5 Step 5 — Supervisor LLM ReAct Reasoning

The Supervisor LLM (GPT-4o-2024-08-06) receives the system prompt loaded from Bedrock Prompt Management, the user question, any episodic memory context, and the list of available tools. It performs ReAct (Reasoning and Acting) reasoning using LangGraph's prebuilt create_react_agent.

The Supervisor's system prompt instructs it to distinguish between three question types and route accordingly:

- Evidence questions (efficacy results, adverse events, dosing, protocol procedures, endpoints) → call research_agent
- Metadata questions (enrollment, status, phase, sponsors, sites, eligibility criteria) → call knowledge_agent
- Visualisation requests (charts, graphs, comparisons of numbers already retrieved) → call chart_agent after getting data

For the CLARITY ARR query the LLM reasons:

```
Thought: The user asks about ARR reduction — an efficacy result from a clinical trial protocol.
This requires retrieving text from the Pinecone corpus. I should call research_agent.

Action: research_agent
Action Input: {"query": "CLARITY trial Cladribine Tablets annualized relapse rate reduction placebo"}
```

2.6 Step 6 — A2A Invocation of the Research Agent

The Supervisor calls invoke_sub_agent() in agents/supervisor/a2a_tools/invoke.py. This function SigV4-signs and sends an HTTP POST to the Research AgentCore runtime. The session ID is namespaced to isolate the Research agent's Postgres conversation history:

```
# In agents/supervisor/a2a_tools/invoke.py
agent_session_id = f'{session_id}__{agent_name}'
# Produces: 'c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b__research'

# The base_session_id (without __agent suffix) is the key for the span buffer.
# It matches the DynamoDB run_id so TraceMiddleware can link spans to trace records.
base_session_id = agent_session_id.split('__')[0]
# Produces: 'c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b'
```

2.7 Step 7 — Research Agent Runs the 4-Stage RAG Pipeline

The Research agent loads its own system prompt from Bedrock (SSM path /research/prod/bedrock/prompt_version — note: research not research-agent), receives the query, and calls the search_tool MCP Lambda followed by the summariser_tool MCP Lambda. The full pipeline is described in Section 3.

2.8 Step 8 — SSE Event Streaming Back to Supervisor

The Research agent streams its response as SSE events. The Supervisor's parse_sse_stream() processes each event type and either forwards it to the UI or stores it for later use:

SSE Event Type	Supervisor Action

type: token	Immediately forwarded to the platform UI via an async token_queue so the user sees the Research agent typing in real-time. The user does not know they are watching a sub-agent — the tokens appear as if the Supervisor is typing.
type: tool_start	Forwarded to UI as a 'searching...' indicator. Tool name is the MCP tool name (e.g. tool-search___search_tool).
type: tool_end	Forwarded to UI to dismiss the indicator.
type: done	The most important event. Contains rag_metrics dict with all Stage 1-4 pipeline measurements. The Supervisor's invoke.py calls _append_span(base_session_id, span_data) to store these metrics for the TracerMiddleware to write to DynamoDB.
type: interrupt	HITLInterrupt is re-raised. Propagates up through the Supervisor's middleware stack where HITLMiddleware catches it and pauses the stream.
type: error	RuntimeError is raised. LangGraph creates a ToolMessage with the error text. The Supervisor LLM sees the error and can either retry or inform the user.

2.9 Step 9 — Supervisor Continues ReAct Loop

The Research agent's answer becomes a ToolMessage in the Supervisor's LangGraph conversation. GPT-4o reads it and reasons again. For simple questions this is the final step — the Supervisor synthesises a response and streams it. For complex queries requiring both graph metadata and clinical evidence, the Supervisor may call knowledge_agent next before synthesising.

2.10 Step 10 — Middleware After-Agent Chain (Reverse Order)

After the Supervisor LLM generates its final answer, the 5 middleware layers execute in innermost-to-outermost order (the reverse of before_agent). Each layer has an after_agent hook:

6. OutputGuardrailMiddleware.after_agent — Sends the answer text to AWS Bedrock Guardrail (ApplyGuardrail API). The guardrail checks three things in sequence: (1) denied topics (OffTopicQuery, PersonalMedicalAdvice), (2) content filters (HATE, SEXUAL, VIOLENCE, MISCONDUCT, PROMPT_ATTACK — all set to HIGH sensitivity), (3) PII detection (NAME and EMAIL are anonymized, SSN is blocked). If blocked, replaces the answer with the blocked output message. Annotates the TracerMiddleware with guardrail_passed, guardrail_blocked, faithfulness_score, and consistency_score.
7. HITLMiddleware.after_agent — No action if no interrupt was triggered.
8. EpisodicMemoryMiddleware.after_agent — Reads the LLM's EPISODIC: YES/NO tag appended to the answer. YES → stores the Q&A pair in Pinecone namespace 'episodic__<user_id>'. NO → discards. The tag is stripped from the answer before it reaches the user by the _stream_events tail buffer in app.py (detailed in Section 5.3). Annotates TracerMiddleware with episodic_hits and episodic_stored.

9. SemanticCacheMiddleware.after_agent — Stores the question embedding and answer text in Pinecone namespace 'cache_pharma' with a 1-hour TTL (3,600 seconds). Uses a background daemon thread to avoid blocking the response. Annotates TracerMiddleware with cache_hit=True if it served a cached answer (otherwise cache_hit defaults to False in the tracer).
10. TracerMiddleware.after_agent — Reads all annotations collected by other middleware via TracerMiddleware.update_trace(). Calculates total elapsed time, aggregates token counts across all LLM turns, computes cost at GPT-4o pricing ($2.50/1M input, $10.00/1M output). Calls pop_span_buffer(session_id) to collect the Research agent RAG metrics from the span buffer. Writes the complete trace to DynamoDB asynchronously in a background thread (ThreadPoolExecutor, 1 worker) so the response is not blocked by the DynamoDB write. Then emits the SSE done event.

2.11 Step 11 — Done Event and UI Completion

```
data: {"type": "done", "latency_ms": 44531.61, "run_id": "c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b"}
```

The platform extracts run_id from the done event. The Observability Dashboard queries DynamoDB with run_id as the primary key and renders the RAG pipeline metrics card showing Stages 1-4 data.

3. The 4-Stage RAG Pipeline in Detail

The Research agent does not call GPT-4o with the raw Pinecone results. It runs a 4-stage quality pipeline that progressively filters, ranks, compresses, and grounds the evidence before the synthesis LLM ever touches it. Each stage is independently measurable via DynamoDB trace fields, which is precisely what makes the harness engineering possible.

3.1 Understanding the Corpus — What Pinecone Actually Contains

The Pinecone clinical-trials-index namespace 'clinical-trials' holds 5,772 chunks derived from 29 clinical trial protocol PDFs. These chunks are not uniform — there are two fundamentally different types, and the difference matters enormously for understanding why Stage 2 thresholds are set where they are.

Type 1 — Verbatim Clinical Trial Text (3 NCTs, very few chunks)

These chunks contain directly extracted sentences from the original protocol documents. They are the highest quality data in the corpus and score 0.95-1.00 on Cohere rerank because the terminology matches exactly. There are only a handful of these across the entire corpus:

```
Chunk: chunk_71c79067a5252c46  |  NCT00213135 (CLARITY trial)

## CLARITY Clinical Trial (NCT00213135)

In the Phase III, multi-center, randomized, double-blind CLARITY clinical trial
(NCT00213135), treatment with Cladribine Tablets demonstrated a significant reduction
in the annualized relapse rate (ARR) at 2 years from 0.33 to 0.14 when compared with
placebo. This corresponded to an ARR risk reduction of 57.4% for the 3.5 mg/kg dose.
The reduction in ARR was also consistent across various patient subgroups, including
those with highly active disease. Furthermore, the CLARITY study demonstrated that
Cladribine Tablets were significantly superior to placebo in the treatment of RRMS
across all clinical and MRI efficacy outcomes (Giovannoni, 2010).
```

Type 2 — AI-Generated Table Descriptions (majority of corpus, 5,700+ chunks)

The majority of the corpus consists of GPT-4-generated descriptions of clinical trial data tables. These were created because tabular data (dose modification tables, schedule of events, statistical summaries) cannot be directly embedded — LLMs cannot extract meaning from raw table HTML. Each table was described in a 7-section format (PURPOSE, STRUCTURE, FINDINGS, STATISTICS, COHORTS, CAVEATS, KEYWORDS). These chunks score 0.15-0.80 on Cohere rerank depending on how closely the query terminology matches the description vocabulary:

```
Chunk: chunk_54f133ae92266efa  |  NCT02788279 (Cobimetinib LVEF)

*Caption:* Table 7 Recommended Dose Modifications for Cobimetinib in Patients with
Left Ventricular Ejection Fraction Decrease from Baseline
```

```
> 1. PURPOSE - The table presents recommended dose modifications for cobimetinib in
> patients experiencing a decrease in Left Ventricular Ejection Fraction (LVEF) from
> baseline values. This clinical guidance aims to address safety concerns related to
> cardiovascular health during treatment with cobimetinib, especially when combined
> with atezolizumab. The endpoint assessed is the LVEF decrease, a critical
> cardiovascular parameter used as a marker for cardiac function deterioration.
>
> 2. STRUCTURE - The table uses rows representing different LVEF decrease thresholds
> and columns for symptomatic and asymptomatic patient categories, providing
> corresponding dose modification recommendations...
```

💡 **WHY THIS MATTERS FOR THRESHOLDS**	A user asking 'What are the dose modifications for cobimetinib LVEF decrease?' uses exactly those words. The AI-generated description also uses exactly those words in section 1 (PURPOSE). Cohere scores this correctly at 0.65-0.75. But a user asking a slightly different phrasing — 'How do I adjust cobimetinib if cardiac function drops?' — uses different vocabulary from the description, scoring 0.20-0.40. That is why the confidence gate is 0.15, not 0.30. Raising the threshold to 0.30 would cause the gate to fire incorrectly on legitimate queries about AI-table-described content.

3.2 Stage 1 — Pinecone Semantic Search

What it does

Converts the user query to a 1,536-dimensional embedding vector using OpenAI's text-embedding-3-small and queries Pinecone by cosine similarity to retrieve the 100 most similar chunks from the corpus.

Why these specific settings

Setting	Value	Why This Value
Embedding model	text-embedding-3-small (1,536 dims)	MUST match the model used during corpus ingestion. Using text-embedding-3-large (3,072 dims) causes a Pinecone dimension mismatch error and returns zero results. The corpus was ingested with text-embedding-3-small.
Index name	clinical-trials-index	Read from SSM at /vs-agentcore-multiagent/prod/pinecone/clinical_trials_index at Lambda cold start.
Namespace	clinical-trials	CRITICAL — the default Pinecone namespace is empty. All 5,772 clinical trial chunks live in this specific named namespace. Omitting the namespace parameter returns exactly 0 results on every query. This was the first production bug discovered.
top_k	100	Bi-encoder embeddings optimise for recall, not precision. The embedding of 'ARR reduction Cladribine' will retrieve

		100 chunks — most of which are marginally related (other MS trials, pharmacokinetics sections, etc.). Casting a wide net gives Cohere's cross-encoder enough candidates to find the single best match even when query and chunk use different vocabulary.

Actual search_lambda code

```
def handler(event, context):
    query = event.get('query', '')
    top_k = event.get('top_k', 100)

    # Embed query — must match ingestion model
    embedding = openai_client.embeddings.create(
        model='text-embedding-3-small',  # 1,536 dims — matches corpus
        input=query,
    ).data[0].embedding

    # Query the correct namespace — this was the production bug
    results = pinecone_index.query(
        vector=embedding,
        top_k=top_k,
        namespace='clinical-trials',   # CRITICAL — not the default empty namespace
        include_metadata=True,
    )
    chunks = [{'id': m['id'], 'score': m['score'], 'text': m['metadata']['text']}
              for m in results.get('matches', [])]
    # rag_chunks_stage1 = len(chunks) — stored in rag_metrics
```

DynamoDB trace field after Stage 1

```
"rag_chunks_stage1": "100"   # 100 chunks retrieved from Pinecone via cosine similarity
```

3.3 Stage 2 — Cohere Rerank (Cross-Encoder)

Why reranking is necessary — bi-encoder vs cross-encoder

Pinecone uses bi-encoder similarity: the query and each chunk are embedded independently and compared as vectors. This is fast (one query embedding compared to millions of pre-computed chunk embeddings) but imprecise. The bi-encoder for 'CLARITY ARR reduction Cladribine' might rank a Cladribine manufacturing quality chunk at position 15 because both share many pharmaceutical terms, even though the manufacturing chunk says nothing about ARR reduction.

Cohere Rerank uses a cross-encoder: it reads the query and each candidate chunk together in a single inference pass, understanding their relationship contextually. It scores the verbatim CLARITY ARR chunk at 1.0 and the manufacturing quality chunk at 0.03. Cross-encoders are

too slow to run against the entire Pinecone index — but perfectly fast for the 100 candidates Stage 1 already narrowed down.

The confidence gate — why 0.15

After reranking, the top Cohere score determines whether there is sufficient evidence in the corpus to answer the query. The threshold 0.15 was calibrated empirically: at 0.15, the gate correctly fires on queries about trials not in the corpus (Pfizer BNT162b2, fake NCT IDs, PACIFIC trial) while correctly passing on legitimate queries about AI-table-described content (e.g. cobimetinib LVEF modifications that score 0.40-0.70). Raising to 0.30 would produce false negatives on legitimate AI-table queries.

```
if top_score < 0.15:  # RERANK_THRESHOLD
    return f'RAG_METRICS:{json.dumps(rag_metrics)}\n\nINSUFFICIENT_CONTEXT: '
        'No chunks in the knowledge base scored above the relevance threshold.'
        'Do not hallucinate an answer. Return this message to the user.'
```

RAG_METRICS transport — why plain text not JSON

The search_tool Lambda returns a plain text string with a machine-readable prefix rather than a JSON object. This design decision was deliberate: the Research agent LLM must call the summariser tool next. If the search result were a JSON object, the LLM would need to parse it and re-format it as a tool argument — LLMs are unreliable at this, often truncating or reformatting large JSON payloads. Plain text with a stripped header keeps the full chunk text intact.

```
# search_lambda return format:
return f"RAG_METRICS:{json.dumps(rag_metrics)}\n\n{formatted_chunks}"

# research_app.py on_tool_end strips the header before the LLM sees the result:
if tool_output.startswith("RAG_METRICS:"):
    lines = tool_output.split('\n\n', 1)
    _rag_metrics.update(json.loads(lines[0][len('RAG_METRICS:'):]))
    tool_output = lines[1]  # Only the chunk text goes to the LLM
```

Real Cohere scores from our production DynamoDB traces

Query	NCT	Top Cohere Score	Interpretation
What was the ARR reduction in the CLARITY trial?	NCT00213135	1.0000	Verbatim text — exact vocabulary match
What were the ORACLE MS safety and efficacy findings?	NCT00725985	0.9971	Verbatim text — very close vocabulary

mRNA-1273 100 µg dose selection rationale	NCT04283461	0.8913	Verbatim text, slightly different phrasing
Cobimetinib LVEF dose modifications	NCT02788279	0.6842	AI table description — good vocab match
LAG-3 positive vs negative DOR and ORR results	NCT03662659	0.5231	AI table description — moderate match
Janssen COVID vaccine safety monitoring schedule	NCT04652245	0.3814	AI table description — loose vocab match
BNT162b2 Pfizer pediatric trial efficacy	None	0.0431	Not in corpus — gate fires correctly

DynamoDB trace fields after Stage 2

```
"rag_chunks_stage2":      "20",     # 20 chunks kept after reranking (top_n=20)
"rag_rerank_top_score":   "1.0",    # highest Cohere relevance score
"rag_rerank_threshold":   "0.15",   # confidence gate threshold
"rag_threshold_triggered": "False",  # gate did NOT fire (good — corpus has evidence)
"rag_top_chunk_ids":      ["chunk_71c79067a5252c46", "chunk_4dd6b0f0516394a5", ...]
"rag_top_chunk_scores":   ["1", "0.9971", "0.9963", "0.9943", "0.9612"]
```

3.4 Stage 3 — GPT-4o-mini Compression

The problem Stage 3 solves

20 reranked chunks at approximately 300 tokens each equals 6,000 tokens of evidence passed to the synthesis LLM. GPT-4o can handle this — its context window is 128K tokens. But the cost is the constraint. Sending 6,000 tokens as input to GPT-4o on every research query would cost approximately $0.015 per query in input tokens alone. GPT-4o-mini costs 30 times less. Stage 3 uses GPT-4o-mini to filter out irrelevant sentences from each chunk, reducing 6,000 tokens to 500-1,500 tokens before the expensive GPT-4o synthesis call.

How compression works

For each of the 20 reranked chunks, GPT-4o-mini receives the original user query and the chunk text. It is instructed to extract only the sentences directly relevant to answering the query. Chunks with no relevant sentences return NOT_RELEVANT and are excluded from the synthesis step.

Concrete example — CLARITY query

Input chunk to Stage 3 (chunk_71c79067a5252c46, 294 tokens):

```
## CLARITY Clinical Trial (NCT00213135)
In the Phase III, multi-center, randomized, double-blind CLARITY clinical trial
(NCT00213135), treatment with Cladribine Tablets demonstrated a significant reduction
in the annualized relapse rate (ARR) at 2 years from 0.33 to 0.14 when compared with
placebo. This corresponded to an ARR risk reduction of 57.4% for the 3.5 mg/kg dose.
```

```
The reduction in ARR was also consistent across various patient subgroups, including
those with highly active disease. Furthermore, the CLARITY study demonstrated that
Cladribine Tablets were significantly superior to placebo in the treatment of RRMS
across all clinical and MRI efficacy outcomes (Giovannoni, 2010).

The CLARITY-EXT study showed that in patients who followed Cladribine Tablets with
placebo, a substantial proportion remained free from indicators of disease activity
during Years 3 and 4 compared to patients who were randomized to further treatment.
[... 180 more tokens about CLARITY-EXT outcomes ...]
```

GPT-4o-mini output after Stage 3 compression (164 tokens — 44% reduction):

```
Treatment with Cladribine Tablets demonstrated a reduction in the annualized relapse
rate (ARR) at 2 years from 0.33 to 0.14 compared with placebo. This corresponded to
an ARR risk reduction of 57.4% for the 3.5 mg/kg dose. The reduction in ARR was
consistent across patient subgroups including those with highly active disease.
```

The CLARITY-EXT sentences were stripped because they discuss continuation therapy outcomes, not the core ARR reduction the query asked about.

Known issue: chunks_input too low

⚠ KNOWN ISSUE — chunks_input: 1-3 instead of 20	The Research agent LLM should pass all 20 reranked chunks to the summariser as a single verbatim text string. Instead, the LLM selects 1-3 chunks it considers most relevant and passes only those. This is a prompt engineering constraint — the LLM cannot reliably pass 20,000 tokens of text as a tool argument without truncating or selecting. Current mitigation: research prompt explicitly instructs 'pass the ENTIRE text returned by search_tool verbatim'. Further work needed.

DynamoDB trace fields after Stage 3

```
"rag_chunks_input":       "3",    # how many chunks the LLM actually passed to summariser
"rag_chunks_compressed":   "2",    # chunks with relevant content after NOT_RELEVANT filtering
"rag_compression_ratio":   "0.667",# tokens retained / tokens input to this stage
"rag_total_tokens_before": "294",  # tokens before compression
"rag_total_tokens_after":  "164",  # tokens after compression
```

3.5 Stage 4 — GPT-4o Citation Synthesis and Hallucination Stripping

What Stage 4 does

GPT-4o receives the compressed chunks from Stage 3 and synthesises a final answer. Every factual claim in the answer must be supported by a [Source: NCT_ID] citation drawn from the retrieved chunks. The summariser_lambda prompt instructs GPT-4o to omit any claim it cannot cite — this is the primary hallucination guard.

Summariser output format

The summariser_lambda returns a JSON envelope (unlike search_lambda which returns plain text). This is parsed by research_app.py to extract rag_metrics from Stage 3+4:

```
{"answer": "Treatment with Cladribine Tablets in the Phase III CLARITY trial...
 [Source: NCT00213135].",
 "rag_metrics": {
   "chunks_input": 3, "chunks_compressed": 2,
   "total_tokens_before": 294, "total_tokens_after": 164,
   "compression_ratio": 0.667, "citation_coverage": 0.667,
   "uncited_claims_stripped": 1
 }
}
```

DynamoDB trace fields after Stage 4

```
"rag_citation_coverage":       "0.667",  # 2 of 3 answer claims have [Source: NCT] citations
"rag_uncited_claims_stripped":   "1",      # 1 claim removed because no chunk supported it
```

3.6 The Span Buffer — Cross-Process RAG Metrics Transport

The Supervisor and Research agents run in separate AgentCore microVMs. They cannot share Python memory. The Supervisor's TracerMiddleware needs the Research agent's RAG metrics (rag_chunks_stage1, rag_rerank_top_score, etc.) to write a complete DynamoDB trace. The span buffer solves this using the SSE done event as the data transport channel.

Component	Location	What It Does
research_app.py on_tool_end	Research agent microVM	Captures RAG_METRICS header from search_tool output. Stores in _rag_metrics dict.
research_app.py (done event)	Research agent microVM	Yields done event with rag_metrics field containing all Stage 1-4 measurements.
parser.py parse_sse_stream	Supervisor microVM	Reads the done event from the SSE stream. Extracts rag_metrics into span_data dict.
invoke.py _append_span	Supervisor microVM	Stores span_data in module-level dict keyed by base_session_id. base_session_id = thread_id without the __agent suffix.
tracer.py after_agent	Supervisor microVM	Calls pop_span_buffer(session_id). Iterates over span data. Writes all rag_* fields to DynamoDB trace record.

```
# The span buffer import path matters — must import from the module, not the package __init__
# WRONG: from agents.supervisor.a2a_tools import pop_span_buffer
# RIGHT: from agents.supervisor.a2a_tools.invoke import pop_span_buffer

# Because _SPAN_BUFFER dict lives in invoke.py module scope, not in __init__.py
```

4. Knowledge Graph Flow — Neo4j in Detail

4.1 When to Use the Knowledge Agent vs the Research Agent

The distinction is structural. The Research agent answers questions whose answers live as natural language text inside protocol PDF documents. The Knowledge agent answers questions whose answers are properties of nodes or traversals of relationships in the graph — numbers, names, dates, counts. A research question never requires traversing a graph edge. A knowledge question never requires reading a paragraph of clinical text.

Question	Agent	Reason
What was the ARR reduction in CLARITY?	Research (Pinecone)	The answer is a percentage from a clinical endpoint analysis — lives in protocol text
How many patients enrolled in NCT04470427?	Knowledge (Neo4j)	t.enrollmentCount is a Trial node property — structured metadata
What adverse events require dose modification for cobimetinib?	Research (Pinecone)	Protocol safety section text — lives in PDF chunks
Which sponsor funds the CLARITY trial?	Knowledge (Neo4j)	(Trial)-[:SPONSORED_BY]->(Sponsor) — graph relationship traversal
What is the schedule of events from Day 64 to Day 394?	Research (Pinecone)	Schedule table description — lives in AI-generated chunk
What primary and secondary outcomes does NCT03662659 measure?	Knowledge (Neo4j)	(Trial)-[:MEASURES]->(Outcome) — 459 structured Outcome nodes
How should Grade 4 CRS be managed during atezolizumab?	Research (Pinecone)	Management protocol text from IRR/CRS safety section
How many sites does NCT03155620 have and in which countries?	Knowledge (Neo4j)	(Trial)-[:LOCATED_AT]->(Site)-[:IN_COUNTRY]->(Country) — graph traversal

4.2 Node Types and Their Properties

Node Type	Count	Key Properties	Common Query Pattern
Trial	25	nctId (indexed unique), briefTitle, officialTitle, acronym, phase, overallStatus, enrollmentCount, enrollmentType, startDate, completionDate, primaryCompletionDate, lastUpdateSubmitDate, statusVerifiedDate	MATCH (t:Trial {nctId:'NCT00213135'}) RETURN t.*

Disease	87	name (indexed unique)	MATCH (t:Trial)-[:TARGETS]->(d:Disease) WHERE d.name CONTAINS 'sclerosis'
Sponsor	23	name (indexed unique)	MATCH (t:Trial {nctId:...})-[:SPONSORED_BY]->(s:Sponsor) RETURN s.name
CRO	42	name (indexed unique)	MATCH (t:Trial)-[:MANAGED_BY]->(cro:CRO) RETURN cro.name
Site	1,102	facility, city, lat, lon, zip	MATCH (t:Trial)-[:LOCATED_AT]->(s:Site) RETURN s.facility, s.city
Country	52	name (indexed unique)	MATCH (s:Site)-[:IN_COUNTRY]->(c:Country) RETURN c.name
Outcome	459	type (Primary/Secondary), measure, description, timeFrame	MATCH (t:Trial)-[:MEASURES]->(o:Outcome) WHERE o.type='Primary' RETURN o.measure
PatientPopulation	25	gender, minimumAge, maximumAge, stdAges (list), eligibilityCriteria (full text), healthyVolunteers	MATCH (t:Trial)-[:INCLUDES]->(p:PatientPopulation) RETURN p.eligibilityCriteria
MeSHTerm	107	term	MATCH (t:Trial)-[:ASSOCIATED_WITH]->(m:MeSHTerm) RETURN m.term
TrialCategory	25	name	MATCH (t:Trial)-[:BELONGS_TO]->(tc:TrialCategory) RETURN tc.name

4.3 Real Cypher Queries the Knowledge Agent Generates

Enrollment and status lookup

```
MATCH (t:Trial {nctId: 'NCT04470427'})
```

```
RETURN t.briefTitle, t.enrollmentCount, t.overallStatus, t.phase, t.startDate, t.completionDate
```

Site distribution by country

```
MATCH (t:Trial {nctId: 'NCT03374254'})
    -[:LOCATED_AT]->(s:Site)-[:IN_COUNTRY]->(c:Country)
RETURN c.name, count(s) AS site_count
ORDER BY site_count DESC
```

Outcome measures for a trial

```
MATCH (t:Trial {nctId: 'NCT03662659'})-[:MEASURES]->(o:Outcome)
RETURN o.type, o.measure, o.description, o.timeFrame
ORDER BY o.type
```

Disease filter with status

```
MATCH (t:Trial)-[:TARGETS]->(d:Disease)
WHERE toLower(d.name) CONTAINS 'multiple sclerosis'
RETURN t.nctId, t.briefTitle, t.phase, t.overallStatus
```

Patient eligibility criteria

```
MATCH (t:Trial {nctId: 'NCT03434379'})-[:INCLUDES]->(p:PatientPopulation)
RETURN p.gender, p.minimumAge, p.maximumAge, p.stdAges,
    p.eligibilityCriteria, p.healthyVolunteers
```

Phase 3 trials ranked by enrollment

```
MATCH (t:Trial)
WHERE t.phase CONTAINS 'Phase 3' OR t.phase CONTAINS 'PHASE3'
RETURN t.nctId, t.briefTitle, t.enrollmentCount
ORDER BY t.enrollmentCount DESC
```

5. Middleware Stack — Implementation Details

5.1 The Middleware Architecture

The Supervisor uses BaseAgentMiddleware from LangChain's middleware system. Layers are registered as a list: [TracerMiddleware, SemanticCacheMiddleware, EpisodicMemoryMiddleware, HITLMiddleware, OutputGuardrailMiddleware]. before_agent executes index 0 to 4. after_agent executes index 4 to 0 (innermost first). This is the standard middleware onion pattern — each layer wraps the next.

5.2 TracerMiddleware — Inter-Middleware Communication

TracerMiddleware uses a class-level annotation registry (_pending_annotations dict) to receive data from other middleware without coupling them together. Any middleware can call TracerMiddleware.update_trace(run_id, {...}) at any point. TracerMiddleware.after_agent() collects all annotations via _pop_annotations(run_id) and merges them into the trace before writing to DynamoDB.

```
# OutputGuardrailMiddleware annotates the tracer after its check completes:
TracerMiddleware.update_trace(run_id, {
   "guardrail_passed":    True,
   "guardrail_blocked":   False,
   "faithfulness_score":  0.92,
   "consistency_score":   1.00,
})

# EpisodicMemoryMiddleware annotates after deciding to store:
TracerMiddleware.update_trace(run_id, {
   "episodic_hits":   3,
   "episodic_stored": True,
})

# SemanticCacheMiddleware annotates on cache hit:
TracerMiddleware.update_trace(run_id, {
   "cache_hit":       True,
   "cache_namespace": "cache_pharma",
})
```

Cost calculation uses GPT-4o pricing ($2.50 per 1M input tokens, $10.00 per 1M output tokens). Token counts come from usage_metadata on each AIMessage. The tracer walks backwards through state['messages'] to find the last AIMessage after each model call and extracts input_tokens and output_tokens from its usage_metadata.

5.3 Episodic Tag Stripping — The Tail Buffer

EpisodicMemoryMiddleware instructs the LLM to append 'EPISODIC: YES' or 'EPISODIC: NO' to every response so after_agent can decide whether to store the Q&A pair in Pinecone. The problem: tokens stream out one at a time. By the time after_agent strips the tag from state['messages'], the tag has already been sent to the user as individual token SSE events:

```
data: {"type":"token","content":"...consult healthcare professionals."}
data: {"type":"token","content":"\n"}
data: {"type":"token","content":"\nEPISODIC:"}
data: {"type":"token","content":" NO"}
```

Fix: the _stream_events function in app.py maintains a _tail_buffer accumulating the last 60 characters of token output. The safe prefix (everything before the last 60 chars) is flushed to the UI immediately. After the stream ends, the tail is cleaned with _EPISODIC_PATTERN before flushing. The tail size of 60 characters safely covers the worst case tag: '\n\nEPISODIC: NO1.01.0' (~22 chars plus leading whitespace and margin).

```
_EPISODIC_PATTERN = re.compile(r'\s*\nEPISODIC:\s*(YES|NO)[\d.\s]*$', re.IGNORECASE)
_TAIL_SIZE = 60

# During streaming — flush safe prefix in real-time:
_tail_buffer += token_content
if len(_tail_buffer) > _TAIL_SIZE:
    safe = _tail_buffer[:-_TAIL_SIZE]
    _tail_buffer = _tail_buffer[-_TAIL_SIZE:]
    yield {'type': 'token', 'content': safe}

# After stream ends — strip tag from tail:
clean_tail = _EPISODIC_PATTERN.sub('', _tail_buffer).rstrip()
if clean_tail:
    yield {'type': 'token', 'content': clean_tail}
```

5.4 SemanticCache — Key Design Decisions

Decision	Value	Reason
Similarity threshold — pharma domain	0.97 cosine similarity	Clinical questions must match very precisely. 0.97 means only near-identical phrasings hit. 'What was the CLARITY ARR?' and 'CLARITY trial ARR reduction percent' will hit. 'What were CLARITY outcomes?' will miss and trigger a fresh retrieval.
Similarity threshold — general domain	0.88 cosine similarity	Less precise questions are more tolerant of paraphrase. General domain is used for non-pharma deployments.
Multi-turn skip	Skip if len(human_messages) > 1	A cached answer from session A cannot be reliably used in session B mid-conversation because the

		conversational context is different. Cache only applies to single-turn (first-message) queries.
Write thread	Background daemon thread	Writing to Pinecone after every response adds 200-400ms latency. A daemon thread runs the write asynchronously after the SSE response completes. daemon=True means container shutdown is not blocked.
TTL	3,600 seconds (1 hour)	Balances cache effectiveness vs clinical data freshness risk. Clinical trial data does not change hour-to-hour, but stale guardrail fallback messages should not be cached permanently.
Never cache guardrail fallbacks	Check for 'did not meet safety and accuracy standards' before writing	Caching a guardrail rejection response would permanently block subsequent legitimate queries that happen to rephrase the blocked query above the 0.97 threshold.

5.5 Bedrock Guardrail — Configuration and Denied Topics

Guardrail Policy	Type	Config	Block Condition / Example
OffTopicQuery	Denied Topic	Enumerated: personal entertainment, sports, weather, financial markets, creative writing, software development with no medical connection	'What is the weather in Boston?' — zero connection to medicine or drug research
PersonalMedicalAdvice	Denied Topic	Direct dosing instructions or recommendations for an individual patient	'What dose of Cladribine should my patient take?' or 'Should I stop my medication?'
HATE	Content Filter	HIGH input + HIGH output sensitivity	Discriminatory language targeting any group
INSULTS	Content Filter	HIGH input + HIGH output	Derogatory or demeaning language
SEXUAL	Content Filter	HIGH input + HIGH output	Sexual content in any context
VIOLENCE	Content Filter	HIGH input + HIGH output	Descriptions of violence or gore

MISCONDUCT	Content Filter	HIGH input + HIGH output	Instructions for illegal activities
PROMPT_ATTACK	Content Filter	HIGH input + NONE output	Prompt injection attempts in user query — output check is None because the agent's own answer cannot be a prompt attack
NAME / EMAIL / PHONE	PII	ANONYMIZE — replace with [REDACTED]	Any personally identifiable information in the answer
US_SOCIAL_SECURITY_NUMBER	PII	BLOCK — reject entire response	SSN in any context

Note: AGE and ADDRESS PII types were initially included but caused false positives — 'participants aged 18 and older' triggered AGE anonymization, and 'patients in the US' triggered ADDRESS. Both were removed from the config after testing.

6. Human-in-the-Loop (HITL) — Four Paths

HITL allows the Supervisor to pause mid-execution and ask the user for clarification before continuing. The implementation is more complex than it appears because LangGraph's interrupt mechanism can surface in four different ways depending on the LangGraph version and runtime configuration. app.py handles all four paths.

6.1 What Triggers HITL

The Supervisor LLM calls the ask_user_input tool when a query is too ambiguous to answer correctly without more context. The options it presents must be real trial names from the corpus — not generic placeholders like 'Trial A' or 'Option 1'. The system prompt explicitly instructs the LLM to search the corpus first and generate options from actual NCT IDs.

- 'Tell me about the cancer trial' — multiple cancer trials in corpus (NCT03662659 LAG-3, NCT03434379 HCC, NCT03374254 CRC)
- 'What were the results?' — no trial specified, no context
- 'Compare the MS trials' — CLARITY, ORACLE MS, and CLASSIC-MS all present in corpus
- 'What is the dose?' — which drug, which trial, which route of administration

6.2 The Four HITL Detection Paths in app.py

When the LLM calls ask_user_input, LangGraph's interrupt mechanism fires. app.py must detect the interrupt regardless of which path it surfaces through — different LangGraph versions use different mechanisms:

Path	Detection Method	When It Fires	Production Priority
Path A	on_tool_end event with HITL tool name	When interrupt_on is NOT active — the ask_user_input tool runs to completion and its result is in the on_tool_end event	Fallback — rarely used in production
Path B	on_chain_end event containing __interrupt__ key in output dict	Some LangGraph versions surface the interrupt as a chain end event	Fallback — version-dependent
Path C	agent.aget_state() checkpoint read after stream ends	PRIMARY PATH in production. When interrupt_on fires, LangGraph intercepts the tool call before on_tool_start. The stream closes with no tokens. The interrupt value is stored in the Postgres checkpoint under state.tasks[0].interrupts[0].	PRIMARY — all production HITL requests go through this path

Path D	GraphInterrupt exception raised by astream_events	Oldest LangGraph versions raise an exception instead of storing in checkpoint	Legacy fallback

6.3 The HITL Resume Sequence

Stage 1 — User receives interrupt event

```
data: {"type": "interrupt",
      "question": "Which multiple sclerosis trial would you like information about?",
      "options": [
        "CLARITY trial (NCT00213135) — Cladribine ARR reduction from 0.33 to 0.14",
        "ORACLE MS trial (NCT00725985) — First clinical demyelinating event",
        "CLASSIC-MS study (NCT03961204) — Long-term MS outcomes and MRI endpoints"
      ],
      "allow_freetext": true
}
```

Stage 2 — User selects an option, UI sends resume message

```
POST /api/v1/clinical-trial/chat
{
  "message":     "[HITL Answer]: CLARITY trial (NCT00213135) — Cladribine ARR reduction. Now search and answer.",
  "thread_id":   "c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b",
  "domain":      "pharma",
  "resume":      true,
  "user_answer": "CLARITY trial (NCT00213135)"
}
```

Stage 3 — Dangling tool call repair in Postgres

When interrupt_on fires, LangGraph stores an AIMessage with tool_calls in the Postgres checkpoint but no corresponding ToolMessage (because the tool was interrupted before completing). OpenAI requires every tool_call to have a matching ToolMessage before a subsequent human turn — without this the API returns: 'An assistant message with tool_calls must be followed by tool messages'. app.py repairs this before resuming:

```
state    = await agent.aget_state(config)
messages = state.values.get('messages', [])

# Find tool_call IDs that have no matching ToolMessage
result_ids = {m.tool_call_id for m in messages if hasattr(m, 'tool_call_id')}
dangling   = [tc for msg in messages
              for tc in getattr(msg, 'tool_calls', [])
              if tc.get('id') not in result_ids]

if dangling:
    await agent.aupdate_state(config, {
        'messages': [ToolMessage(
```

```
        content=f'[Interrupted — user answered: {user_answer}]',
        tool_call_id=tc['id'],
        name=tc.get('name', 'unknown'),
    ) for tc in dangling]
})
```

Stage 4 — Agent resumes with user answer

The resume payload injects '[HITL Answer]: CLARITY trial (NCT00213135)...' as the next human message. HITLMiddleware.before_agent detects the prior HITL ToolMessage in conversation history and injects a SystemMessage: 'The user has already answered the clarifying question — do not call ask_user_input again.' The agent calls search_tool with the specific trial name and returns a grounded answer.

7. Observability — Complete DynamoDB Trace Schema

7.1 A Complete Real Trace Record

Here is an actual DynamoDB trace record from our production system for the CLARITY ARR query. Every field is populated and annotated:


```
{
  // Identity
  "run_id":    "c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b",   // Primary key = thread_id
  "ts":       1746187234.918,                            // Unix timestamp
  "user_id":   "c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b",   // Same as run_id in this deployment
  "session_id":"c73e7373-abc1-4d2e-b8f1-9a0e3d5c2f7b",
  "domain":    "pharma",
  "prompt_version": "19",                                 // Research agent prompt version
  "observability":  "langsmith",                          // LangSmith run tree attached

  // Request content
  "question":      "What was the ARR reduction in the CLARITY trial?",
  "answer":        "In the Phase III CLARITY trial (NCT00213135), treatment with Cladribine
                   Tablets reduced the ARR from 0.33 to 0.14 [Source: NCT00213135].",
  "answer_length":  247,
  "is_resume":      false,
  "hitl_user_answer": "",

  // Performance
  "elapsed_ms":     44531.61,
  "llm_timings":    [{"turn":1,"elapsed_ms":1203,"input_tokens":2841,"output_tokens":67},
                    {"turn":2,"elapsed_ms":8291,"input_tokens":3214,"output_tokens":312}],
  "cache_hit":      false,
  "cache_namespace": null,

  // Tokens and cost
  "input_tokens":   6055,
  "output_tokens":  379,
  "total_tokens":   6434,
  "token_cost_usd": 0.018938,                             // $2.50/1M input + $10/1M output

  // Tool routing
  "tools":       ["research_agent"],
  "tool_count":  1,
  "llm_turns":   2,                                       // 2 LLM calls: routing + synthesis
  "tool_details": [{"name":"research_agent",
                   "args_summary":"CLARITY trial Cladribine ARR reduction placebo",
                   "result_summary":"In the Phase III CLARITY...",
                   "is_error":false}],

  // HITL
```

```
  "hitl_fired":    false,
  "hitl_question": "",
  "hitl_options":  [],

  // Guardrail
  "guardrail_passed":    true,
  "guardrail_blocked":   false,
  "faithfulness_score":  0.97,
  "consistency_score":   1.00,

  // Memory
  "episodic_hits":   0,
  "episodic_stored": true,                          // LLM tagged EPISODIC: YES

  // Errors
  "has_errors": false,
  "errors":     [],

  // RAG pipeline — from Research agent span buffer
  "rag_chunks_stage1":       "100",
  "rag_chunks_stage2":       "20",
  "rag_rerank_top_score":    "1.0",
  "rag_rerank_threshold":    "0.15",
  "rag_threshold_triggered":  "False",
  "rag_top_chunk_ids":       ["chunk_71c79067a5252c46", "chunk_4dd6b0f0516394a5",
                              "chunk_ac860883c12a92c7", "chunk_6da50d4b18410ff7",
                              "chunk_08eac67e452ccbbf"],
  "rag_top_chunk_scores":    ["1", "0.9971", "0.9963", "0.9943", "0.9612"],
  "rag_chunks_input":        "3",
  "rag_chunks_compressed":    "2",
  "rag_compression_ratio":    "0.667",
  "rag_total_tokens_before":  "294",
  "rag_total_tokens_after":   "164",
  "rag_citation_coverage":    "0.667",
  "rag_uncited_claims_stripped": "1"
}
```

8. Harness Engineering — What It Is and How We Built It

8.1 What Is Harness Engineering?

In traditional software, you write a unit test that asserts: function X with input Y produces output Z. The test is deterministic. AI systems cannot be tested this way — a large language model answering 'What was the ARR reduction in CLARITY?' might word the answer differently every time, might include or exclude a specific sentence, and might cite the source as 'NCT00213135' or 'the CLARITY trial' interchangeably. Yet it can still be correct or incorrect.

Harness engineering is the discipline of building an automated test suite that runs real end-to-end queries against the live system and validates correctness across multiple independent dimensions simultaneously. Each dimension is a signal. No single dimension proves the system works — but together they provide a strong composite signal.

For this platform, a single query is validated across 10 dimensions:

Dimension	What It Measures	How It Is Checked	What Failure Means
Stage 1 Recall	Did Pinecone retrieve enough candidate chunks?	rag_chunks_stage1 >= 50 from DynamoDB trace	Namespace bug, wrong embedding model, or index issue — Stage 1 returned too few candidates for Stage 2 to work with
Stage 2 Rerank Quality	Did Cohere rank enough chunks above minimum quality?	rag_chunks_stage2 >= 5 from DynamoDB trace	Reranker configuration issue or wrong Cohere model version
Confidence Gate Status	Did the gate fire correctly for this query type?	rag_threshold_triggered == False (positive), True (negative)	Gate fired on a valid query (false negative) or missed a negative query (false positive — hallucination risk)
Minimum Rerank Score	Was the top Cohere score at least the expected minimum for this query?	rag_rerank_top_score >= q_def['min_rerank_score']	Wrong chunks retrieved, or corpus lacks

		(0.9 for verbatim, 0.4 for AI-table)	coverage for this query
Stage 3 Compression	Did Stage 3 produce compressed output?	rag_chunks_compressed >= 1	Summariser tool called without query arg, or all chunks scored NOT_RELEVANT
Citation Coverage	What fraction of answer claims are cited?	rag_citation_coverage >= 0.30	Hallucination risk — LLM making uncited claims in synthesis step
Expected Chunk Retrieved	Are the known-correct chunk IDs in the top 5 results?	At least one expected_chunk_id in rag_top_chunk_ids	Wrong chunks promoted to top — retrieval precision failure
Keyword Coverage	Does the answer mention the expected terminology?	50% of expected_answer_contains strings found in answer text	Answer exists but uses wrong vocabulary, or answer is too short / truncated
Fact Coverage	Does the answer contain specific expected facts?	40% of expected_facts strings found in answer text	Correct direction but missing key data points (e.g. answer says 'ARR reduced' but omits '0.33 to 0.14')
NCT Citation	Does the answer cite the expected trial by NCT ID?	expected_nct string found in answer	Answer about wrong trial, or citation system broken

8.2 The Golden Dataset — 49 Verified Queries

The golden dataset is the harness's source of truth. Every query was manually derived from two sources: the Pinecone corpus (pinecone_dump.json, 5,772 chunks) and the Neo4j schema (10 node types, 11 relationships). No query is guesswork — every expected_chunk_id was extracted from pinecone_dump.json and verified to contain the expected facts. Every expected Cypher relationship was verified against the actual Neo4j schema.

Category	Count	Selection Criteria	DynamoDB Fields Validated
RAG High Confidence	8	Queries about NCTs with verbatim text chunks (CLARITY, ORACLE	All 10 rag_* fields

		MS, mRNA-1273). Expected min_rerank_score >= 0.9. These should always pass Stage 1-4 with strong metrics.	
RAG Medium Confidence	10	Queries about NCTs with only AI-generated table descriptions. Expected min_rerank_score 0.40-0.70. Tests Cohere's ability to handle vocabulary mismatches.	All 10 rag_* fields
RAG Negative	5	Queries about trials not in corpus (Pfizer BNT162b2, fake NCT99999999, PACIFIC trial, SUSTAIN-6). Gate MUST fire. Any positive result means hallucination.	rag_threshold_triggered (must be True)
RAG Comparative	4	Multi-trial queries requiring cross-NCT retrieval (CLARITY vs ORACLE MS, atezolizumab vs pembrolizumab irAE). Tests whether multiple expected_chunk_ids appear in top results.	rag_top_chunk_ids (multiple NCTs)
Graph Queries	12	One query per Neo4j relationship type plus aggregate queries. Tests Cypher generation and Knowledge agent routing. Covers all 10 node types.	Content checks — no rag_* fields (Knowledge agent does not use Pinecone)
Cross-Agent	4	Queries requiring Supervisor to call both Knowledge and Research agents. Tests routing logic and answer synthesis across multiple data sources.	tools field (must contain both agents)
HITL	3	Vague queries that should trigger ask_user_input ('Tell me about the cancer trial'). Tests clarification detection.	hitl_fired field
Guardrail	3	Personal medical advice and off-topic queries. Must return blocked output message.	guardrail_blocked field

8.3 Five Real Bugs the Harness Caught

Bug 1 — Pinecone Default Namespace

✕ SYMPTOM	harness.py --quick shows rag_chunks_stage1: 0 on all 3 RAG high queries. Gate fires on every query. All answers: 'I cannot find information about this trial in our knowledge base.'

Without the harness this pattern would be invisible. A human tester would see 'No information found' and assume the corpus lacked coverage for those trials. The harness shows the exact same failure across all 3 queries — pointing to infrastructure, not content. The DynamoDB trace has rag_chunks_stage1: 0, which can only happen if Pinecone returns zero matches. Zero matches across 3 wildly different queries means the index or namespace is wrong, not the query.

```
# Root cause found in search_lambda:
results = index.query(vector=embedding, top_k=100, include_metadata=True)
# Missing: namespace='clinical-trials'
# The default namespace is empty — all 5,772 chunks live in 'clinical-trials'

# Fix:
results = index.query(vector=embedding, top_k=100,
    namespace='clinical-trials',
    include_metadata=True)
# Redeploy lambdas. All 3 harness queries now show chunks_stage1: 100.
```

Bug 2 — Embedding Dimension Mismatch

✕ SYMPTOM	After namespace fix: rag_error field appears in DynamoDB trace — 'Vector dimension 3072 does not match the dimension of the index 1536'. chunks_stage1: 0 still.

The harness run_id pointed to a specific DynamoDB record. The rag_error field (populated by the search_lambda exception handler) contained the exact Pinecone error message. Without the harness writing this to DynamoDB, the error would have been in CloudWatch logs — findable, but requiring manual log searching. The harness surfaces it in the score output immediately.

```
# Root cause: search_lambda was using the wrong embedding model
embedding = openai.embeddings.create(model='text-embedding-3-large', ...)
# text-embedding-3-large returns 3072-dimensional vectors
# The index was built with text-embedding-3-small (1,536 dimensions)
# Pinecone rejects the dimension mismatch

# Fix:
embedding = openai.embeddings.create(model='text-embedding-3-small', ...)
```

Bug 3 — Missing query Argument in Summariser Call

✕ SYMPTOM	Stage 1: 100, Stage 2: 20, Stage 3: 0 compressed chunks. All answers are empty strings. DynamoDB rag_chunks_compressed: 0.

Stages 1 and 2 working but Stage 3 returning zero means the summariser tool is being called but producing no output. The harness run_id links to LangSmith where the full LangGraph trace

shows the exact tool call. The LLM called the summariser with chunks=[...] but no query argument. The summariser_lambda returned 'No query provided — cannot compress chunks without knowing what to look for.' GPT-4o-mini cannot determine relevance without the query.

```
# Old research agent prompt (caused bug):
'Step 2: Call tool-summariser___summariser_tool with the chunks you extracted.'
# LLM passed: {chunks: [...]} — missing query

# Fixed research agent prompt:
'Step 2: Call tool-summariser___summariser_tool with TWO required arguments:
 - chunks: pass the ENTIRE text returned by search_tool verbatim as a string
 - query:  the EXACT query string you used in Step 1
 Both arguments are REQUIRED. Never call summariser without query.'
```

Bug 4 — SSM Path Mismatch (Prompt Version Not Found)

❌ **SYMPTOM**	All research agent calls return empty tools list in DynamoDB. tools: []. Research agent crashes on every invocation. CloudWatch shows: ResourceNotFoundException: PromptVersion 6 doesn't exist.

The harness showed tools: [] on every query — the research_agent tool was being called but the research_agent itself was crashing before returning any SSE events. The harness run_id pointed to the Supervisor DynamoDB trace. Cross-referencing with the Research agent CloudWatch log group (found by listing log groups with prefix /aws/bedrock-agentcore/runtimes/vs_agentcore_ma_research) showed the exact error: prompt version 6 was deleted from Bedrock Prompt Management but SSM still pointed to it.

```
# Root cause: deploy.py was writing to the wrong SSM path
app_name    = f'{agent}-agent'                # 'research-agent'
ssm_ver_key = f'/{app_name}/prod/bedrock/prompt_version'
# Writes to: /research-agent/prod/bedrock/prompt_version

# But AGENT_NAME env var in AgentCore runtime is 'research' (not 'research-agent')
# So the agent reads from: /research/prod/bedrock/prompt_version
# These are two different SSM parameters. The read path had version 6 (deleted).
# The write path had version 17 but the agent never reads it.

# Fix: deploy.py prompts step now uses /{agent}/prod/... not /{agent}-agent/prod/...
ssm_ver_key = f'/{agent}/prod/bedrock/prompt_version'
# Writes to: /research/prod/bedrock/prompt_version — matches AGENT_NAME=research
```

Bug 5 — Negative Query Validation

✅ **EXPECTED BEHAVIOUR**	5 negative queries all show rag_threshold_triggered: True. Answers contain 'INSUFFICIENT_CONTEXT'. Gate fired correctly on queries about Pfizer BNT162b2, fake NCT99999999, PACIFIC trial, SUSTAIN-6, and durvalumab.

This was not a bug — it was a validation. The 5 negative queries confirmed that the confidence gate at 0.15 correctly rejects queries about trials absent from the corpus. During calibration, we tested threshold values from 0.05 to 0.40. At 0.30, three legitimate queries about AI-table-described content (cobimetinib LVEF, LAG-3 endpoints, Janssen vaccine schedule) also fired the gate incorrectly. At 0.15, all 5 negative queries fire the gate and all 18 positive queries pass. The harness is what made this calibration measurement possible.

8.4 Complete Scoring Algorithm

harness.py scores each query independently based on its category. The base score is (checks_passed / total_checks). A query passes if score >= 0.70. The harness exits with code 0 if all queries pass, code 1 if any fail.

```
def score_rag(q_def, answer, trace):
    checks = {}
    is_negative = q_def.get('should_gate_fire', False)

    if is_negative:
        # Negative queries: gate MUST fire
        checks['gate_fired'] = trace.get('rag_threshold_triggered') == 'True'
        kw = q_def.get('expected_answer_contains', [])
        checks['keywords'] = sum(1 for k in kw if k.lower() in answer.lower()) / max(len(kw),1) >= 0.5
    else:
        # Positive queries: all 10 dimensions
        checks['stage1']  = float(trace.get('rag_chunks_stage1', 0)) >= 50
        checks['stage2']  = float(trace.get('rag_chunks_stage2', 0)) >= 5
        checks['gate']    = str(trace.get('rag_threshold_triggered')) == 'False'
        checks['stage3']  = float(trace.get('rag_chunks_compressed', 0)) >= 1
        checks['cite']    = float(trace.get('rag_citation_coverage', 0)) >= 0.30
        checks['score']   = float(trace.get('rag_rerank_top_score', 0)) >= q_def.get('min_rerank_score', 0.15)
        ids = trace.get('rag_top_chunk_ids', [])
        checks['chunks']  = any(c in ids for c in q_def.get('expected_chunk_ids', []))
        kw = q_def.get('expected_answer_contains', [])
        kwhits = [k for k in kw if k.lower() in answer.lower()]
        checks['keywords'] = len(kwhits) / max(len(kw),1) >= 0.5
        facts = q_def.get('expected_facts', [])
        fhits = [f for f in facts if f.lower() in answer.lower()]
        checks['facts']   = len(fhits) / max(len(facts),1) >= 0.40
        checks['nct']     = q_def.get('expected_nct','') in answer

    passed = sum(1 for v in checks.values() if v)
    return {'checks': checks, 'score': passed / len(checks), 'passed': passed, 'total': len(checks)}
```

9. Deploy Operations — Complete Reference

9.1 Deployment Order and Dependencies

Every step in deploy.py reads SSM parameters written by earlier steps. The order is not arbitrary — deploying out of order will fail because later steps reference resources that do not yet exist in SSM.

Step	Command	What It Does	Reads From SSM	Writes to SSM
1	deploy.py secrets	Pushes all API keys to Secrets Manager. Writes platform_api_key to both /vs-agentcore-multiagent/prod/ (for agents) and /vs-agentcore/prod/ (for Terraform ECS injection). Writes postgres credentials after RDS is deployed.	OPENAI_API_KEY, PINECONE_API_KEY etc. from .env.prod	SSM_PREFIX/ SSM_PREFIX/ SSM_PREFIX/
2	deploy.py iam	Creates 3 IAM roles: lambda-mcp (Lambda execution), gateway-role (MCP Gateway), agent-role (AgentCore runtimes). Attaches inline policies with minimum required permissions.	Account ID from STS	IAM role ARN by name)
3	deploy.py guardrails	Creates Bedrock Guardrail with denied topics, content filters, and PII config. Versions it. If guardrail exists, updates content and creates new version.	SSM_PREFIX/bedrock/guardrail_id (if exists)	SSM_PREFIX/ SSM_PREFIX/
4	deploy.py lambdas	Builds all 5 Lambda Docker images for linux/amd64, pushes to ECR, creates or updates Lambda functions. Sets SSM_PREFIX environment variable on each function.	None	ECR repositor deployed_res
5	deploy.py gateway	Creates MCP Gateway or updates existing. Creates/updates 5 gateway targets (tool-search, tool-graph, tool-summariser, tool-chart, tool-hitl/clarify). Adds Lambda resource-based permission policies.	SSM_PREFIX/mcp/gateway_url (if exists)	SSM_PREFIX/
6	deploy.py prompts	Creates or updates Bedrock prompts for all 5 agents. Reads prompt text from prompts/{agent}.txt. Writes to /{agent}/prod/bedrock/prompt_id and /{agent}/prod/bedrock/prompt_version. Deletes oldest version if at 10-version limit.	/{agent}/prod/bedrock/prompt_id (if exists)	/{agent}/proc /{agent}/proc
7	deploy.py registry	Writes the agent registry JSON to SSM. The registry defines which agents the Supervisor can route to (research,	None	SSM_PREFIX/

		knowledge, chart — safety excluded). Supervisor reads this at cold start.		
8	deploy.py agents	Builds all 5 agent Docker images for linux/arm64 (REQUIRED — x86_64 images will not run on AgentCore). Pushes to ECR. Creates or updates AgentCore runtimes. Deploys supervisor LAST because it reads sub-agent ARNs from SSM.	SSM_PREFIX/mcp/gateway_url, /{agent}/prod/bedrock/*, SSM_PREFIX/agents/*/runtime_arn	SSM_PREFIX/
9	deploy.py platform	Builds FastAPI and UI Docker images for linux/amd64. Runs terraform apply. Force-deploys ECS services to latest task definition. Sets ALB idle timeout to 300s.	SSM_PREFIX/platform_api_key, RDS_PASSWORD env var	ECS task defi (tracked by Te

9.2 Critical SSM Paths

SSM Path	Value	Read By
/research/prod/bedrock/prompt_id	44F06K0CNE	Research age get_bedrock_
/research/prod/bedrock/prompt_version	DRAFT or version number e.g. 19	Research age invocation (n
/supervisor/prod/bedrock/prompt_id	66UDJ8VW3W	Supervisor a
/vs-agentcore-multiagent/prod/agents/supervisor/runtime_arn	arn:aws:bedrock-agentcore:...:runtime/vs_agentcore_ma_supervisor-...	Platform app.
/vs-agentcore-multiagent/prod/agents/research/runtime_arn	arn:aws:bedrock-agentcore:...:runtime/vs_agentcore_ma_research-...	Supervisor a2
/vs-agentcore-multiagent/prod/agents/registry	JSON array of agent definitions	Supervisor co StructuredTo
/vs-agentcore-multiagent/prod/pinecone/clinical_trials_index	clinical-trials-index	search_lambd at cold start
/vs-agentcore-multiagent/prod/mcp/gateway_url	https://...bedrock-agentcore.us-east-1.amazonaws.com/...	All agent run MCP Gatewa
/vs-agentcore-multiagent/prod/dynamodb/trace_table_name	vs-agentcore-ma-traces	TracerMiddle
/vs-agentcore-multiagent/prod/bedrock/guardrail_id	Guardrail resource ID	OutputGuard start
/vs-agentcore/prod/platform_api_key	Platform API key (SecureString)	ECS Fargate t Terraform pa

9.3 Common Production Issues and Fixes

Issue	Root Cause	Fix	Prevention
ResourceNotFoundException: PromptVersion N doesn't exist	deploy.py wrote to /{agent}-agent/prod/ but agent reads from /{agent}/prod/ — different SSM paths. Old version deleted from Bedrock.	aws ssm put-parameter --name /research/prod/bedrock/prompt_version --value DRAFT --overwrite	deploy.py now uses /{agent}/prod/ paths. Set to DRAFT to always load latest without version tracking.
rag_chunks_stage1: 0 on every query	search_lambda querying default (empty) Pinecone namespace instead of 'clinical-trials'	Add namespace='clinical-trials' to index.query() call. Redeploy lambdas.	Always explicitly set namespace in Pinecone queries — never rely on default.
rag_error: dimension mismatch 3072 vs 1536	search_lambda using text-embedding-3-large (3072 dims) but corpus ingested with text-embedding-3-small (1536 dims)	Change embedding model to text-embedding-3-small in search_lambda.	Document the embedding model in SSM. Validate on first Lambda deployment.
tools: [] — research_agent never called	AgentCore microVM stuck on deleted prompt version cached in old container	Wait 15 minutes for microVM idle timeout, or delete and recreate the AgentCore runtime via AWS Console (no CLI delete command in current SDK)	Remove @lru_cache from all SSM and Bedrock reads. Always read fresh.
rag_chunks_compressed: 0	LLM calling summariser with chunks argument but omitting query argument	Update research prompt to explicitly require both arguments with example	Add validation in summariser_lambda to return error if query is missing
502 Bad Gateway from ALB	ALB idle timeout (60s default) shorter than agent response time (30-90s)	Set ALB idle_timeout.timeout_seconds to 300 via modify_load_balancer_attributes	Set this in Terraform resource or deploy.py platform step post-apply

pop_span_buffer returns empty dict	tracer.py importing pop_span_buffer from package __init__ instead of invoke module	Import from agents.supervisor.a2a_tools.invoke not from agents.supervisor.a2a_tools	The _SPAN_BUFFER dict lives in invoke.py scope. __init__.py re-exports don't carry the dict reference.

10. Running the Harness — Complete Reference

10.1 Setup

```
cd ~/PycharmProjects/vs-agentcore-multiagent

# Golden dataset — source of truth for all harness validation
mkdir -p tests
cp ~/Downloads/golden_dataset.json tests/golden_dataset.json

# Harness script
cp ~/Downloads/harness.py scripts/harness.py

# Environment — PLATFORM_API_KEY and AWS credentials must be set
source scripts/.env.prod
echo $PLATFORM_API_KEY | head -c 8  # verify key loaded
aws sts get-caller-identity         # verify AWS credentials
```

10.2 Command Reference

Command	Queries Run	Use Case	Approximate Time
harness.py --quick	3 rag_high + 2 graph + 1 rag_negative = 6 queries	Post-deploy smoke test — confirms the system is alive and pipeline is working	5-8 minutes
harness.py --category rag_high	8 queries	Validate Pinecone + Cohere + GPT-4o-mini + GPT-4o pipeline after prompt changes	10-15 minutes
harness.py --category rag_negative	5 queries	Confirm confidence gate fires correctly — critical after threshold changes	6-10 minutes
harness.py --category graph	12 queries	Validate Neo4j Knowledge agent routing and Cypher generation	15-20 minutes
harness.py --category guardrail	3 queries	Confirm Bedrock Guardrail blocks personal medical advice and off-topic queries	3-5 minutes
harness.py --category cross_agent	4 queries	Validate Supervisor routes to both Knowledge + Research agents in same query	8-12 minutes
harness.py --category hitl	3 queries	Confirm vague queries trigger ask_user_input clarification	4-6 minutes

harness.py --id R01 G04 N02	3 specific queries	Debug a specific failing query by ID	2-4 minutes
harness.py --output results.json	All 49 queries	Full regression run — save results for comparison	60-90 minutes
harness.py --report	No API calls	Show last 10 DynamoDB traces with RAG metrics — diagnose without running queries	Instant
delete_cache.py --stats	No API calls	Show count of cached entries before clearing	Instant
delete_cache.py	No API calls	Delete all semantic cache entries — forces fresh retrieval for next harness run	10-30 seconds

10.3 Sample Passing and Failing Output

Passing query (R01 — CLARITY ARR, score 90%)

```
────────────────────────────────────────────────────────────────────────────────
[RAG_HIGH] R01 — ✅ PASS (90%)
Query: What was the annualized relapse rate reduction with Cladribine Tablets...
Latency: 44.5s  run_id: c73e7373...

Checks:
  ✅ Pinecone recall >=50:          100
  ✅ Cohere rerank kept >=5:         20
  ✅ Top score >=0.9:                1.0
  ✅ Gate not fired:                 False
  ✅ Stage 3 output >=1:             2
  ✅ Citations >=30%:                0.667
  ✅ Expected chunk retrieved:       chunk_71c79067a5252c46 in top 5
  ✅ Keyword coverage >=50%:         0.83  hits=['relapse rate','0.33','0.14','57.4']
  ❌ Fact coverage >=40%:            0.25  hits=['0.33','0.14']  missing=['57.4%','3.5 mg/kg']
  ✅ NCT citation:                   NCT00213135 found
```

Failing query (N01 — Pfizer BNT162b2, gate should fire)

```
[RAG_NEGATIVE] N01 — ❌ FAIL (50%)
Query: What were the Phase 3 efficacy results for the BNT162b2 Pfizer COVID vaccine...
Latency: 8.2s  run_id: f829a41e...

Checks:
  ✅ Gate fired (expected):          True — rag_threshold_triggered = True
  ❌ Keyword coverage >=50%:         0.00  hits=[]  — answer lacks 'not' / 'cannot'
      Answer: 'The BNT162b2 vaccine demonstrated 95% efficacy...'  ← HALLUCINATION
```

```
⚠️ CRITICAL: Gate fired correctly (score 0.11 below 0.15 threshold)
⚠️ BUT: Agent ignored INSUFFICIENT_CONTEXT and hallucinated an answer
⚠️ Action required: Research agent prompt must enforce returning INSUFFICIENT_CONTEXT
```

10.4 Using the Harness as a Deployment Gate

The harness exit code (0 = pass, 1 = fail) makes it suitable as an automated deployment gate. After any change to prompts, Lambda code, or agent configuration:

```
# After prompt change:
python3.11 deploy.py prompts

# Run critical categories — fail fast
python3.11 harness.py --category rag_high --category rag_negative
if [ $? -ne 0 ]; then
  echo 'REGRESSION DETECTED — reverting prompt version'
  aws ssm put-parameter \
    --name /research/prod/bedrock/prompt_version \
    --value <previous_version> --type String --overwrite
  exit 1
fi

# After full agent redeploy:
python3.11 harness.py --quick
if [ $? -ne 0 ]; then
  echo 'POST-DEPLOY SMOKE TEST FAILED'
  exit 1
fi
echo 'Deployment validated. Running full harness in background...'
python3.11 harness.py --output results_$(date +%Y%m%d_%H%M%S).json &
```